

LabResults API Platform Foundation

1. Overview

This repository provisions the AWS platform foundation for the LabResults API: a Python-based backend serving web traffic. The design targets a small two-person engineering team, prioritizing low operational overhead, fast delivery, strong security defaults, and high availability.

The solution is delivered as reusable Terraform modules (vpc, alb, ecs, observability) composed in an environment root (envs/prod), a GitHub Actions pipeline that builds, scans, and deploys the container, and a CloudWatch-based observability layer (logs, dashboard, alarms).

Stack summary

Layer	Choice
Compute	AWS ECS Fargate
Networking	Custom VPC module with 2 AZs, public + private subnets, IGW and a NAT Gateway per AZ
Load balancing	Application Load Balancer, HTTP + optional HTTPS listener, S3 access logs
Container registry	Amazon ECR, immutable tags, scan-on-push
CI/CD	GitHub Actions, OIDC federation
IAM	Dedicated execution role, task role, and CI role, least privilege, scoped resources
Observability	CloudWatch Logs, CloudWatch Dashboard, CloudWatch Alarms + SNS (email)
IaC	Terraform, modular (terraform/modules/*), environment root at terraform/envs/prod

For design rationale, trade-offs, assumptions, and future work, see the companion document DECISIONS.md.

2. Architecture

Clients access the application through the Internet Gateway and then connect to the Application Load Balancer, which is deployed across two public subnets in two Availability Zones.

The Application Load Balancer forwards incoming requests to a target group containing ECS Fargate tasks. These tasks run in private subnets across two Availability Zones and receive traffic on the configured container port.

The ECS tasks use a NAT Gateway in each Availability Zone to access the internet for outbound traffic only. This allows the tasks to pull external dependencies or communicate with external APIs without exposing the tasks directly to the internet.

The ECS tasks can also access Amazon ECR through the VPC routing configuration and IAM permissions. They send logs to CloudWatch using the ECS execution role, and they can access other AWS services based on the permissions granted to their task role.

For deployments, GitHub Actions authenticates with AWS by assuming an IAM role through OIDC. This means there are no long-lived AWS access keys stored in GitHub. GitHub Actions then builds and pushes the container image to Amazon ECR and updates the ECS service to deploy the new image.

Component reference

Component	AWS Service	Purpose
Network isolation	VPC (2 AZs)	Isolation boundary; nothing reachable without explicit routing/SG rules
Public subnets	Subnets x2	Host only the ALB and NAT Gateway ENIs
Private subnets	Subnets x2	Host ECS Fargate tasks without public Ips
Edge routing	Internet Gateway	Ingress/egress for public subnets
Outbound egress	NAT Gateway (per AZ)	Private subnet tasks reach the internet without inbound exposure
Load balancing	Application Load Balancer	Terminates client traffic, forwards to healthy ECS tasks; access logs to S3
Compute	ECS Fargate cluster + service	Runs the LabResults API container; autoscales 2–10 tasks on request count
Registry	ECR repository	Stores built images; immutable tags, scan-on-push, least-privilege repo policy
CI/CD identity	IAM OIDC provider + role	GitHub Actions assumes a scoped role, no static credentials in the repo
Task identities	ECS execution role / task role	Execution role pulls images & writes logs; task role is empty by default, extended per app need

Component	AWS Service	Purpose
Observability	CloudWatch Logs, Dashboard, Alarms, SNS	Centralized logs, a health dashboard, and 5xx alerting

3. Prerequisites

- AWS account/credentials with permission to create VPC, IAM, ECS, ECR, ALB, CloudWatch, and SNS resources
- Terraform >= 1.x installed locally
- AWS CLI v2 configured
- Environment variables / AWS credentials exported in your terminal so Terraform and the AWS CLI can authenticate. See below to get the access keys:
set AWS_ACCESS_KEY_ID=<your-access-key-id>
set AWS_SECRET_ACCESS_KEY=<your-secret-access-key>
set AWS_DEFAULT_REGION=<region>
- A GitHub repository for the application code. Empty repository with access.

Procedure to get access keys:

1. Log in to the **AWS Management Console**.
2. Click your account name in the top-right corner and select **Security credentials**
3. Scroll down to the **Access keys** section.
4. Click **Create access key**.
5. Select your use case and check the acknowledgment box.
6. Click **Next**, then **Create access key**.
7. Copy the **Access Key ID** and **Secret Access Key** immediately.
8. AWS Access Key ID starts with **Starts with: AKIA..** AWS Secret Access Key is a Random base64 characters.
9. For the Region, look at the top-right corner of the AWS Management Console next to your username.
10. Note the location listed (e.g., *N. Virginia, Frankfurt, US East (N. Virginia)*).
11. Use the corresponding AWS region code (e.g., us-east-1, eu-central-1, us-west-2).
12. Set the three variables using (in cmd)
13. set AWS_ACCESS_KEY_ID=<your-access-key-id>
14. set AWS_SECRET_ACCESS_KEY=<your-secret-access-key>
15. set AWS_DEFAULT_REGION=<region>

4. Infrastructure as Code (Terraform)

The Terraform is organized as reusable modules under terraform/modules/ (vpc, alb, ecs, observability) composed by an environment root at terraform/envs/prod.

Module	Responsibility
modules/vpc	VPC, subnets, routing, NAT, IGW, flow logs
modules/alb	Load balancer, listeners, target group, security group, access-log bucket

Module	Responsibility
modules/ec2	Cluster, task definition, service, autoscaling, task security group
modules/observability	Dashboard, alarms, SNS topic

4.1 First-time deploy

Create a new folder "source_files"

From the terminal enter the folder and clone the source repository with the terraform and application files.

git clone <https://github.com/jcastro2943/lab-results-api-inf.git>

enter the folder lab-results-api-inf/terraform/envs/prod

On the very first apply, the ECS task definition requires a container image reference before the actual image is pushed by the pipeline. Use a public NGINX placeholder image, as shown in the command below (in cmd). This parameter is fixed.

For the other two variables, retrieve the OIDC subject claim from GitHub Actions. Open the repository on GitHub and go to **Settings** → **Actions** → **OIDC**. Under **Default subject claim prefix**, locate the value shown for the repository's OIDC subject claim prefix. Copy only the value after repo: prefix (discard repo:) place it after github_repository= as shown in the example below.

Finally, replace the email address in the command with a working email address.

Run the 2 commands for the last command replace the values and copy the entire command. Enter yes when prompted to start the deployment of the infrastructure.

```
terraform init

terraform apply ^ -var="container_image=public.ecr.aws/nginx/nginx:latest" ^ -
var="github_repository=myrepouser@34338838/myrepo@134683433" ^ -
var="alarm_notification_emails=["example@gmail.com"]"
```

Key variables:

Variable	Purpose	Notes
container_image	Image URI for the ECS task	Required. Use a placeholder on first apply; CI/CD updates it on later deploys.

Variable	Purpose	Notes
github_repository_OIDC	org/repo of the app source	OIDC subject claim to establish trust policy
alarm_notification_emails	Email for SNS alerts	An automatic email will be sent. The holder must confirm its SNS subscription via email clicking the link.

4.2 Verification and Outputs

If terraform apply succeeds with no errors, Terraform prints an Outputs: block. Copy the following outputs:

- The dashboard_url is the URL of the monitoring dashboard, while the alb_dns_name is the URL used to access the deployed application. Copy the alb_dns_name value now, as it will be used later to open the application and validate that the deployment is working correctly.
- After a successful terraform apply, Terraform outputs the following values: ecr_repository_url, github_actions_ci_role_arn, ecs_cluster_name, ecs_service_name, ecs_task_definition_family, and container_name. These outputs are required by the CI/CD pipeline. Copy the values from the Terraform output and use them in the next step when configuring the GitHub Actions workflow.

4.3 Subsequent applies

Once the CI/CD pipeline is wired up (see Section 5), infrastructure changes go through the same terraform apply, while application image updates are handled by the pipeline's ECS deploy step, not by re-running Terraform.

5. CI/CD

Pipeline flow, stages, and triggers

The workflow (.github/workflows/pipeline.yaml) triggers on: push to main, tags matching v*.*.*, pull requests targeting main, and manual workflow_dispatch.

Job	Runs on	AWS access	What it does
build-and-scan	All PRs and main	None (unprivileged)	Builds the Docker image locally, scans it with Trivy (SARIF uploaded to the GitHub Security tab), fails the

			build on CRITICAL vulnerabilities, and saves the image as an artifact
push-to-ecr	main / tags / dispatch only	OIDC — id-token: write scoped to this job only	Downloads the scanned image artifact, authenticates to AWS, logs into ECR, tags and pushes the image
deploy-ecs	main / tags / dispatch only	OIDC — id-token: write scoped to this job only	Fetches the current task definition, renders it with the new image URI, and deploys to ECS with wait-for-service-stability enabled

Pull requests only run the build-and-scan job — no AWS credentials are ever exposed to a PR, which limits blast radius from untrusted or external contributions. id-token: write is scoped only to the two jobs that need it, not the whole workflow.

Deployment strategy

The image is delivered via **ECS rolling deployment**: deploy-ecs fetches the live task definition, replaces only the image field for the named container, registers a new task definition revision, and calls ecs update-service. ECS then launches new tasks (up to 200% of desired count), waits for them to pass both the ALB health check and the container health check, and only then drains and stops old tasks (never dropping below 100% of desired count). The workflow blocks (wait-for-service-stability: true) until these finishes.

Rollback strategy

Rollback is largely automatic: the ECS **deployment circuit breaker** detects tasks that fail to reach a steady, healthy state and automatically reverts the service to the previous task definition revision, no manual action needed for a bad deploy that fails health checks.

Implementation

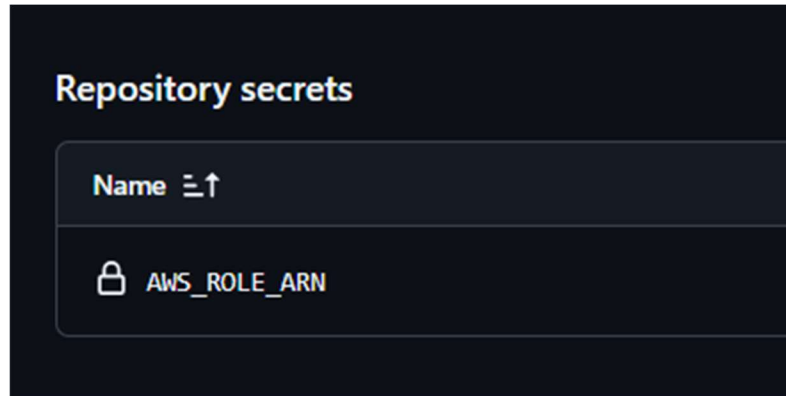
1. In the Terraform output, copy the CI/CD-relevant values as explained in the previous section. The following table is an example, could change depending on the output of terraform.

Terraform variable	Repository variable	Value
container_name	CONTAINER_NAME	lab-api-prod
ecr_repository_url	ECR_REPOSITORY	lab-api-prod
ecs_cluster_name	ECS_CLUSTER	lab-api-prod-cluster
ecs_service_name	ECS_SERVICE	lab-api-prod-service
ecs_task_definition_family	ECS_TASK_DEFINITION_NAME	lab-api-prod-task
	AWS_REGION	Depend on your account (us-east-1)

2. AWS_ROLE_ARN will hold the value of github_actions_ci_role_arn
3. In your **application** GitHub repository → Settings → Secrets and variables → Actions:
 - Add **Repository secret**: AWS_ROLE_ARN = the github_actions_ci_role_arn Terraform output.
 - Add **Repository variables**: AWS_REGION, ECR_REPOSITORY, CONTAINER_NAME, ECS_CLUSTER, ECS_SERVICE, ECS_TASK_DEFINITION_NAME, from the matching Terraform outputs.

The result should look like this (variables could change depending on terraform's output). This is an example:

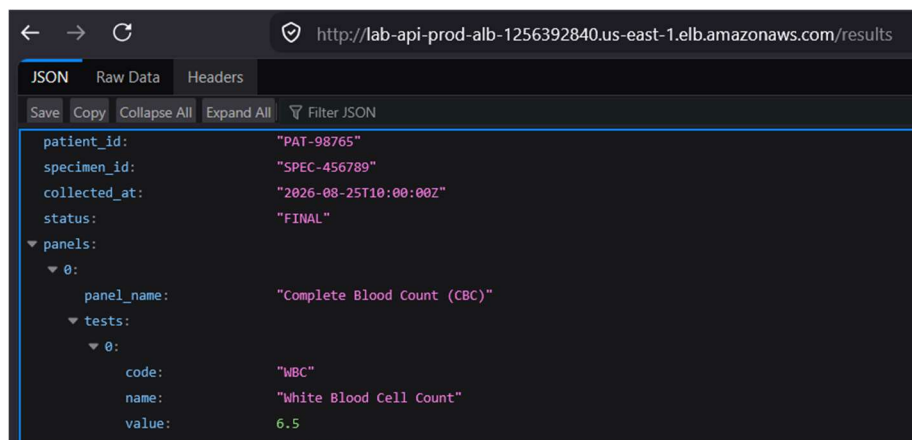
Repository variables	
Name 	Value
 AWS_REGION	us-east-1 
 CONTAINER_NAME	myapp-prod 
 ECR_REPOSITORY	myapp-prod 
 ECS_CLUSTER	myapp-prod-cluster 
 ECS_SERVICE	myapp-prod-service 
 ECS_TASK_DEFINITION_NAME	myapp-prod-task 



4. From your local machine, create a new folder "backend" in a separate location (outside of "source_files" folder) and copy application/ folder's contents into the new backend folder. This is the application and pipeline source code. Run the following commands.
5. `git init`
6. `git add .`
7. `git commit -m "Initial commit"`
8. `git branch -M main`
9. `git remote add origin https://github.com/USERNAME/REPOSITORY.git` (this is an empty repository you created and have access to, from prerequisites) and run:

`git push -u origin main`

10. Go to the repository's **Actions** tab, the pipeline starts automatically on the push. It typically completes in a few minutes; watch each job go green in sequence (build-and-scan → push-to-ecr → deploy-ecs).
11. Once deploy-ecs finishes, open in the browser the `alb_dns_name` from the Terraform output and append `/results`, you should get a JSON response with exam data. As shown in the following image



6. Observability

The following observability stack is implemented in terraform:

Tool	Lives at	Shows
CloudWatch Dashboard	CloudWatch console → Dashboards → lab-api-dashboard (URL is a direct Terraform output)	ALB request count, target response time (p50/p90/p99), HTTP status code breakdown (2xx/4xx/target-5xx/ELB-5xx), healthy/unhealthy target count, ECS CPU %, ECS memory %, running vs desired task count, and a live Logs Insights query showing the last 20 error/exception/fatal log lines
Application container logs	CloudWatch Logs → log group /ecs/lab-api	stdout/stderr from the application container, one log stream per task
VPC Flow Logs	CloudWatch Logs → log group /aws/vpc-flow-logs/ lab-api	All accepted/rejected network traffic in the VPC — useful for diagnosing connectivity/security-group issues
ALB access logs	S3 bucket lab-api-alb-access-logs-<account_id>	Every request the ALB received, including client IP, latency, and response code — for detailed forensic/traffic analysis
Alarms	CloudWatch → Alarms	Target 5xx count and ELB 5xx count, each ≥ threshold within 60s
Alerts	SNS topic lab-api-alerts	Emails subscribers when an alarm fires or clears

How to access

- **Dashboard:** Use the dashboard_url Terraform output to open the CloudWatch dashboard directly. Alternatively, go to **AWS Console** → **CloudWatch** → **Dashboards** → **lab-api-dashboard**.
- **Container logs:** Go to **AWS Console** → **CloudWatch** → **Log groups** → **/ecs/lab-api** and select a log stream for the relevant ECS task. For live logs from the CLI, run `aws logs tail /ecs/lab-api --follow`.
- **VPC Flow Logs:** Go to **AWS Console** → **CloudWatch** → **Log groups** → **/aws/vpc-flow-logs/lab-api**. Use **Logs Insights** to filter and investigate accepted or rejected network traffic.
- **ALB access logs:** Open the S3 bucket lab-api-alb-access-logs-<account_id> and browse the ALB log objects. These logs can be downloaded or queried for detailed request and traffic analysis.
- **Alarms:** Go to **AWS Console** → **CloudWatch** → **Alarms** to view the current status and history of the 5xx alarms.
- **Email alerts:** Accept the subscription confirmation by email. Alternatively, subscribe your email address to the lab-api-alerts SNS topic. After subscribing, confirm the subscription using the confirmation email sent by AWS. Once confirmed, you will receive notifications when an alarm **fires or clears**.

Example troubleshooting flow

Scenario: API requests are returning intermittent 5xx errors in production.

1. Start with the CloudWatch Dashboard

- Open lab-api-dashboard using the dashboard_url Terraform output.
- Check **ALB request count** and **HTTP status code breakdown**.
- Confirm whether 5xx responses are coming from the **target (ECS application)** or the **ALB itself**.
- Check **p50/p90/p99 target response time** to see whether the issue is also causing increased latency.
- Check **ECS CPU/memory utilization** and **running vs desired task count**.

2. Determine whether the problem is application-side or infrastructure-side

- **Target 5xx increasing:** investigate the ECS application and its logs.
- **ELB 5xx increasing:** investigate the ALB, target health, networking, or connectivity.
- **High latency + high CPU/memory:** investigate resource exhaustion or an application performance issue.
- **Running tasks below desired count:** investigate ECS task failures, deployments, or health-check failures.

3. Check application container logs

- Go to **CloudWatch** → **Log groups** → **/ecs/lab-api**.
- Open the log streams for the affected ECS tasks.
- Use Logs Insights to search for ERROR, Exception, or FATAL.
- Correlate the timestamps of the errors with the increase in 5xx responses on the dashboard.

4. Check ECS task and target health

- If tasks are restarting or disappearing, check whether they are failing health checks or crashing.
- Compare the **healthy/unhealthy target count** on the dashboard with the ECS task count.
- If targets become unhealthy shortly before the 5xx spike, investigate the application health endpoint, startup time, or container failures.

5. Check VPC Flow Logs if connectivity is suspected

- Open **CloudWatch** → **Log groups** → **/aws/vpc-flow-logs/lab-api**.
- Look for **REJECT** traffic between the affected resources.
- This can help identify issues such as an incorrect security-group rule, blocked database connection, or unexpected network path.

6. Check ALB access logs for request-level evidence

- Open the lab-api-alb-access-logs-<account_id> S3 bucket.
- Review requests around the incident timestamp.
- Use the client IP, request path, latency, and response codes to identify whether the problem affects:
 - all endpoints,
 - a specific endpoint,
 - specific clients, or

- a specific period of time.
- 7. **Check CloudWatch Alarms and SNS notifications**
 - Open **CloudWatch** → **Alarms** and check the target 5xx and ELB 5xx alarms.
 - Review when the alarm entered ALARM and when it returned to OK.
- 8. **Identify and fix the root cause**
 - Example finding: application logs show database connection timeout.
 - VPC Flow Logs show rejected traffic from the ECS task subnet to the database.
 - This points to a **network/security-group configuration issue**, rather than an ALB or application-code problem.
 - Correct the security-group rule and redeploy/apply the infrastructure change.
- 9. **Verify recovery**
 - Return to the CloudWatch dashboard.
 - Confirm that:
 - target 5xx responses have returned to normal,
 - p90/p99 latency has recovered,
 - targets are healthy,
 - ECS running tasks match the desired count, and
 - the CloudWatch alarm has returned to OK.
 - Check the application logs again to confirm the original errors have stopped.

Alarms/improvements to add in production

- **Latency alarm** on TargetResponseTime p99 exceeding an SLO.
- **Unhealthy host count alarm** (currently only visible on the dashboard, not alarmed on).
- **ECS CPU/memory saturation alarms** as an early warning before autoscaling maxes out at 10 tasks.
- **NAT Gateway error/port-allocation alarms** (silent egress failures are hard to spot otherwise).
- **Deployment failure alarm** hooked to ecs:CodeDeploy/EventBridge for immediate deploy-failure notification, rather than relying on someone noticing the dashboard.
- **Budget/cost anomaly alarms**, since autoscaling to 10 tasks or NAT data-processing charges can spike cost quickly.
- **AWS X-Ray** (or OpenTelemetry + AWS Distro for OTel) for request tracing across the ALB → ECS → any downstream dependency, to pinpoint *where* latency or errors originate.
- **Structured JSON logging** in the application (instead of plain text) so CloudWatch Logs Insights queries can filter/aggregate on fields like request_id, status_code, exam_id instead of regex over free text.
- A dedicated observability platform (Datadog, Grafana Cloud, New Relic) if the team needs cross-service correlation, SLO tracking, or better on-call workflows than native CloudWatch alarms provide.

Validation steps

- terraform apply completed with no errors and printed all outputs.
- AWS secrets/variables copied into the application GitHub repo.
- Pipeline run is green across all three jobs in the Actions tab.
- `http://<alb_dns_name>/results` returns exam JSON data.
- Dashboard URL loads and widgets populate with data (confirms metrics wiring).
- Test SNS email subscription is confirmed and an alarm test (e.g. temporarily lowering the 5xx threshold) triggers a real email. The API has an `/error` endpoint to simulate 500 errors, request this endpoint repeatedly to trigger the alert.
- CloudWatch Logs show application log lines from the running tasks.